



Προστασία και Ασφάλεια Υπολογιστικών και Τηλεπικοινωνιακών Συστημάτων — 3η εργασία προς παράδοση

Στοιχεία φοιτητή

Ονοματεπώνυμο: Φωτόπουλος Γεώργιος Κωνσταντίνος

A.M.: 1117202200234



Ολόκληρο το παρόν PDF το έχω κάνει με χρήση του **Notion**.

Κώδικας

CSS

style.css

```
/**  
 * css/style.css  
 *  
 * Full Name: George Constantine Fotopoulos
```

```
* R.N.: 1117202200234
*/

body {
  margin: 16px;
  font-size: 16px;
  font-family: Arial, sans-serif;
}

form {
  width: 360px;
}

label {
  display: block;
  margin-top: 12px;
  margin-bottom: 4px;
}

input, button {
  font-size: inherit;
}

input[type="text"], input[type="password"] {
  width: 100%;
  max-width: 260px;
}

button {
  margin-top: 12px;
  margin-right: 8px;
}

.message-area {
  margin-bottom: 16px;
}
```

```

.message-area p {
    margin-top: 0;
    margin-bottom: 8px;
}

.result-item {
    margin-bottom: 20px;
}

.product-table {
    border-collapse: collapse;
    margin-top: 16px;
}

.product-table th, .product-table td {
    border: 1px solid black;
    padding: 6px 10px;
    text-align: left;
}

```

JavaScript

login_validation.js

```

/**
 * js/login_validation.js
 *
 * Full Name: George Constantine Fotopoulos
 * R.N.: 1117202200234
 */

function ValidateLoginForm() {
    const username_input = document.getElementById("username");
    const password_input = document.getElementById("password");
}

```

```
if (username_input.value.trim() === "" || password_input.value.trim() === "") {  
    alert("Please fill in both the username and password fields.");  
    return false;  
}  
  
return true;  
}
```

PHP

common.php

index.php

vulnerable_login.php

SQL

schema.sql

```
-- sql/schema.sql  
--  
-- Full Name: George Constantine Fotopoulos  
-- R.N.: 1117202200234  
  
--  
-- 1. Setup the tables
```

```
--
```

```
DROP TABLE IF EXISTS Product;  
DROP TABLE IF EXISTS Users;
```

```
--
```

```
-- 2. Create the tables
```

```
--
```

```
CREATE TABLE Users (  
    username VARCHAR(30) PRIMARY KEY,  
    password VARCHAR(255) NOT NULL,  
);
```

```
CREATE TABLE Product (  
    product_id INTEGER GENERATED ALWAYS AS IDENTITY PR  
    IMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    price NUMERIC(10, 2) NOT NULL  
);
```

```
--
```

```
-- 3. Insert the users and
```

```
--
```

```
INSERT INTO Users (username, password)  
VALUES  
    ("mort", "pizza"),  
    ("bob", "bob123"),  
    ("admin", "admin");
```

```
INSERT INTO Product (product_name, price)  
VALUES  
    ("Laptop", 2899.99),  
    ("Mouse", 59.99),
```

```
("Keyboard", 119.99),  
("Monitor", 379.99);
```

reset_products.sql

```
-- sql/reset_products.sql  
--  
-- Full Name: George Constantine Fotopoulos  
-- R.N.: 1117202200234  
  
-- Remove all product rows and reset the identity counter back to 1.  
TRUNCATE TABLE Product RESTART IDENTITY;  
  
INSERT INTO Product (product_name, price)  
VALUES  
    ("Laptop", 2899.99),  
    ("Mouse", 59.99),  
    ("Keyboard", 119.99),  
    ("Monitor", 379.99);
```

Αναφορά

Σκοπός της εργασίας

Στην παρούσα εργασία υλοποιείται ένα παράδειγμα παραβίασης ελέγχου πρόσβασης με χρήση SQL injection (σε PostgreSQL).

Αρχικά, ο χρήστης συμπληρώνει τα πεδία `username` και `password` μέσω της HTML φόρμας.

Αν τα στοιχεία αυτά υπάρχουν στον πίνακα `Users`, τότε εμφανίζονται τα δεδομένα του πίνακα `Product`.

Κανονική λειτουργία

Τα παρακάτω έγκυρα στοιχεία χρήστη:

```
Username: mort  
Password: pizza
```

μας δίνει το αποτέλεσμα:

```
Access granted.
```

και εμφανίζεται ο πίνακας με τα προϊόντα.

Για μη έγκυρα στοιχεία χρήστη:

```
Username: mort  
Password: ablalabla
```

μας δίνει το αποτέλεσμα:

```
Access denied.
```

SQL Injection

Υπόθεση 1

Ο κακόβουλος χρήστης γνωρίζει το username ενός έγκυρου χρήστη, αλλά όχι τον συνθηματικό του· για παράδειγμα:

```
Username: alice  
Password: ' OR '1'='1
```

όπου η ευάλωτη SQL εντολή γίνεται:

```
SELECT username  
FROM Users
```

```
WHERE username = 'alice'
AND password = '' OR '1'='1'
```

δηλαδή, η συνθήκη `'1'='1'` είναι πάντα αληθής.

Άρα, το αποτέλεσμα του ελέγχου ταυτότητας επιστρέφει την εγγραφή από τον πίνακα `Users` που αντιστοιχεί στο δοθέν όνομα χρήστη, και η εφαρμογή θεωρεί ότι ο χρήστης έχει δικαίωμα πρόσβασης.

Με αυτόν τον τρόπο εμφανίζονται τα δεδομένα του πίνακα `Product`, χωρίς ο χρήστης να γνωρίζει το πραγματικό συνθηματικό.

Υπόθεση 2

Ο κακόβουλος χρήστης δε γνωρίζει κανένα έγκυρο username, για παράδειγμα:

```
Username: ' OR '1'='1' --
Password: anything
```

όπου η ευάλωτη SQL εντολή γίνεται:

```
SELECT username
FROM Users
WHERE username = '' OR '1'='1' -- '
AND password = 'anything'
```

δηλαδή το `--` στην PostgreSQL ξεκινά ένα σχόλιο μέχρι το τέλος της γραμμής.

Άρα, το υπόλοιπο κομμάτι της SQL εντολής αγνοείται:

```
AND password = 'anything'
```

κι άρα, ουσιαστικά η συνθήκη γίνεται:

```
username = '' OR '1'='1'
```

για τον ίδιο λόγο ότι το `'1'='1'` είναι πάντα αληθές, το query επιστρέφει τουλάχιστον μία γραμμή από τον πίνακα `Users`.

Στον PHP κώδικα, η εντολή `pg_fetch_assoc($result)` δεν επιστρέφει `false`, οπότε η εφαρμογή θεωρεί ότι ο χρήστης έχει δικαίωμα πρόσβασης.

Έτσι εμφανίζονται τα δεδομένα του πίνακα `Product`, χωρίς ο χρήστης να γνωρίζει ούτε το όνομα χρήστη ούτε το συνθηματικό.

Extra

Έλεγχος ορθότητας των αρχείων

Ο έλεγχος ορθότητας των αρχείων για συντακτικά σφάλματα μπορεί να γίνει με χρήση των παρακάτω εντολών:

```
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\common.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\index.php
C:\xampp\php\php.exe -l C:\xampp\htdocs\asgmt3\vulnerable_log
in.php
```